

FIPS 197

Advanced Encryption Standard (AES)

한국어 학습 번역본

NIST FIPS 197-upd1
원문 발표: 2001-11-26 / Update 1: 2023-05-09

학습 주석

이 문서는 학습과 구현 검증을 위한 비공식 한국어 번역본이다. AES, Block, Byte, State, Key, Round, S-box, XOR, 함수명(예: SUBBYTES()), 변수명, 수학적식, hexadecimal 상수 등은 원문 표기를 유지한다. 두 번째 첨부분의 편집 방식을 기준으로 수식, 행렬, 표, 배치를 수학 조판 형태로 재구성했다.

학습 주석

inverse는 “역변환/역연산”이라는 뜻이다. 어떤 변환의 결과에 그 inverse를 적용하면 원래 값으로 되돌아간다. 예: SHIFROWS()의 inverse는 INVSHIFROWS()이다.

서문 (Foreword)

National Institute of Standards and Technology(NIST)의 Federal Information Processing Standards Publication Series는 법률에 따라 개발·발행되는 표준 및 지침의 공식 publication series이다. 이 FIPS publication에 관한 의견은 공고 부분의 문의 연락처를 통해 제출할 수 있다.

초록 (Abstract)

2000년 NIST는 Advanced Encryption Standard(AES) competition의 winner로 Rijndael block cipher family를 선정했다고 발표하였다. Block cipher는 많은 cryptographic service의 기반이며, 특히 data confidentiality를 보장하는 service에서 핵심적으로 사용된다.

이 표준은 Rijndael family 중 AES-128, AES-192, AES-256 세 가지 instance를 규정한다. 세 algorithm 모두 data를 128-bit block 단위로 변환하며, 이름 뒤의 숫자는 cryptographic Key의 bit length를 나타낸다.

Keywords: AES; block cipher; confidentiality; cryptography; encryption; Rijndael.

AES 표준 공고 사항

1. **Name of Standard.** Advanced Encryption Standard(AES), FIPS 197.
2. **Category.** Computer Security Standard, Cryptography.
3. **Explanation.** AES는 electronic data를 보호하는 FIPS-approved cryptographic algorithm이며 symmetric block cipher이다. 128, 192, 256-bit Key를 사용하고 data는 128-bit block 단위로 처리한다.
4. **Implementation.** software, firmware, hardware 또는 조합으로 구현할 수 있으며 FIPS-approved 또는 NIST-recommended mode of operation과 함께 사용한다.
5. NIST는 implementation이 표준과 일치하는지 시험하는 validation program을 운영한다.
6. 이 표준은 2002년 5월 26일부터 효력이 발생했다.
7. NIST는 분석과 기술 발전을 반영하여 AES를 계속 재평가한다.

학습 주석

AES 그 자체는 “128-bit Block 하나를 Key로 변환하는 primitive”이다. 저장장치에 AES-XTS를 적용할 때는 AES primitive 위에 XTS mode가 추가된다. AES와 AES-XTS는 같은 개념이 아니다.

1 Introduction

Block은 주어진 고정 길이의 bit sequence이다. Block cipher는 Key라고 부르는 bit sequence에 의해 parameterized되는 block permutation family이다.

1997년 NIST는 AES 개발 작업을 시작했고, 2000년 Rijndael을 AES로 선정했다. 이 표준은 Rijndael의 세 가지 instance인 AES-128, AES-192, AES-256을 규정한다. 숫자는 Key의 bit length를 나타낸다. 세 경우 모두 block size는 128 bit이다.

Algorithm	Key length	Block size	Rounds
AES-128	128 bit	128 bit	10
AES-192	192 bit	128 bit	12
AES-256	256 bit	128 bit	14

학습 주석

AES-256의 “256”은 Block 크기가 아니라 Key 길이이다. AES-256도 Block은 128 bit이다.

2 Definitions

2.1 Terms and Acronyms

용어	정의
AES	Advanced Encryption Standard.
Affine transformation	matrix multiplication 뒤에 vector addition이 이어지는 변환.
Array	integer index로 element를 식별하는 고정 크기 data structure.
Bit	binary digit: 0 또는 1.
Block	고정 길이 bit sequence. AES에서는 128 bit.
Block cipher	Key에 의해 parameterized되는 block permutation family.
Byte	8 bit sequence.
Equivalent inverse cipher	CIPHER()의 inverse를 CIPHER()와 유사한 구조로 표현한 대안 specification. 수정된 key schedule을 사용한다.
Key	block cipher family에서 사용할 permutation을 결정하는 parameter.
Key schedule	KEYEXPANSION()으로 생성되는 Round Key sequence.
Rijndael	NIST가 AES competition의 winner로 선정한 block cipher.
Round	CIPHER(), INVCIPHER(), EQINVCIPHER()에서 반복되는 State transformation sequence.
Round Key	Key expansion으로 얻는 $N_r + 1$ 개의 4-word array 중 하나.
State	AES block cipher의 intermediate result. 4-by-4 Byte array.
S-box	byte를 one-to-one substitution하는 non-linear substitution table.
Word	32 bit group = 4 Byte.

2.2 List of Functions

Function	설명
ADDROUNDKEY()	Round Key를 State와 결합하는 transformation.
AES-128()/AES-192()/AES-256()	각 Key length에 해당하는 AES block cipher.
CIPHER()	AES forward block transformation.
INVCIPHER()	CIPHER()의 inverse.
EQINVCIPHER()	modified Key schedule을 사용하는 equivalent inverse cipher.
MIXCOLUMNS()/INVMIX-COLUMNS()	State column mixing / 그 inverse.
SBOX()/INVSBOX()	S-box substitution / inverse substitution.
SHIFTRWS()/IN-VSHIFTRWS()	State row cyclic shift / inverse shift.
SUBBYTES()/INVSUBBYTES()	각 Byte에 S-box / inverse S-box 적용.
KEYEXPANSION()	Key에서 Round Key들을 생성.
KEYEXPANSIONEIC()	equivalent inverse cipher용 modified Round Key 생성.
ROTWORD()	Word의 4 Byte cyclic rotation.
SUBWORD()	Word의 4 Byte 각각에 S-box 적용.
XTIMES()	Byte polynomial에 x 를 곱하고 $m(x)$ 로 modulo reduction.

2.3 Algorithm Parameters and Symbols

Symbol / Parameter	설명
b^{-1}	$GF(2^8)$ element b 의 multiplicative inverse.
$\tilde{b}$	AES S-box affine transformation 입력.
dw	equivalent inverse cipher에 입력되는 Key schedule용 Word array.
$GF(2)$	2개의 element를 갖는 finite field.
$GF(2^8)$	256개의 element를 갖는 finite field.
in / out	CIPHER()/INVCIPHER()의 16-Byte input / output array.
$m(x)$	Byte를 $GF(2^8)$ polynomial로 표현할 때 사용하는 modulus.
key	AES Key를 구성하는 N_k Word array.
N_b	State column 수. AES에서는 4.

Symbol / Parameter	설명
N_k	Key를 구성하는 Word 수: 4, 6, 8.
N_r	Round 수: 10, 12, 14.
$Rcon$	Round constant Word array.
$state$	16 Byte의 2D array.
w	Key schedule Word array.
$\oplus$	XOR.
$\bullet$	$GF(2^8)$ multiplication.
$\leftarrow$	pseudocode assignment.

3 Notation and Conventions

3.1 Inputs and Outputs

Bit는 0 또는 1이다. AES block cipher의 input과 output은 모두 128-bit Block이다. AES-128, AES-192, AES-256의 Key length는 각각 128, 192, 256 bit이다.

3.2 Bytes

AES의 기본 processing unit은 Byte, 즉 8-bit sequence이다. Byte는 $\{b_7b_6b_5b_4b_3b_2b_1b_0\}$ 로 쓰며, 예를 들어 $\{10100011\} = \{a3\}_{16}$ 이다.

4-bit	0000	0001	0010	0011	0100	0101	0110	0111
Hex	0	1	2	3	4	5	6	7
4-bit	1000	1001	1010	1011	1100	1101	1110	1111
Hex	8	9	a	b	c	d	e	f

3.3 Indexing of Byte Sequences

$8k$ 개의 bit sequence가

$$r_0 \ r_1 \ r_2 \ \cdots \ r_{8k-3} \ r_{8k-2} \ r_{8k-1} \quad (3.1)$$

로 주어지면, $0 \leq j \leq k-1$ 에서 Byte a_j 는

$$a_j = \{r_{8j} \ r_{8j+1} \ \cdots \ r_{8j+7}\}. \quad (3.2)$$

으로 정의한다.

학습 주석

전체 bit sequence의 index는 왼쪽에서 0부터 증가하지만, 개별 Byte 내부 bit index는 왼쪽 MSB가 7이다. 두 indexing 규칙을 혼동하면 안 된다.

3.4 The State

AES 내부연산은 4-by-4 Byte 2D array인 State에서 수행한다. Input array $in_0, \dots, in_{15}$ 는

$$s[r, c] = in[r + 4c], \quad 0 \leq r < 4, \ 0 \leq c < 4 \quad (3.6)$$

로 State에 복사하고, 최종 State는

$$out[r + 4c] = s[r, c] \quad (3.7)$$

로 output에 복사한다.

$$\begin{bmatrix} in_0 & in_4 & in_8 & in_{12} \\ in_1 & in_5 & in_9 & in_{13} \\ in_2 & in_6 & in_{10} & in_{14} \\ in_3 & in_7 & in_{11} & in_{15} \end{bmatrix}$$

학습 주석

State는 row-major가 아니라 input을 column 방향으로 채우는 mapping이다. RTL/FPGA에서 byte ordering이 틀리면 계산식이 맞아도 NIST ciphertext와 일치하지 않는다.

3.5 Arrays of Words

Word는 4 Byte이고, 128-bit Block 하나는 4 Word로 구성된다. State의 각 column 하나가 Word 하나로 해석된다.

4 Mathematical Preliminaries

State의 각 Byte는 256-element finite field인 $\text{GF}(2^8)$ 의 element로 해석된다. Byte $\{b_7 \dots b_0\}$ 는

$$b(x) = b_7x^7 + b_6x^6 + b_5x^5 + b_4x^4 + b_3x^3 + b_2x^2 + b_1x + b_0 \quad (4.1)$$

로 해석한다. 예를 들어 $\{01100011\} = x^6 + x^5 + x + 1$ 이다.

4.1 Addition in $\text{GF}(2^8)$

Addition은 polynomial coefficient를 modulo 2로 더하는 것으로 bitwise XOR와 같다.

$$\{57\} \oplus \{83\} = \{d4\}. \quad (4.2)$$

학습 주석

$\text{GF}(2)$ 에서는 $1 + 1 = 0$ 이므로 addition과 subtraction이 동일하다. 일반 정수 산술과 다르다.

4.2 Multiplication in $\text{GF}(2^8)$

두 Byte를 polynomial로 곱은 뒤 다음 fixed polynomial로 modulo reduction한다.

$$m(x) = x^8 + x^4 + x^3 + x + 1. \quad (4.3)$$

Byte b, c 에 대해

$$b \bullet c \longleftarrow b(x)c(x) \bmod m(x). \quad (4.4)$$

특히 $\{02\}$ 를 곱하는 연산은

$$\text{XTIMES}(b) = \begin{cases} \{b_6b_5b_4b_3b_2b_1b_0\}, & b_7 = 0, \\ \{b_6b_5b_4b_3b_2b_1b_0\} \oplus \{1b\}, & b_7 = 1. \end{cases} \quad (4.5)$$

예를 들어

$$\{57\} \bullet \{02\} = \{ae\}, \quad \{57\} \bullet \{04\} = \{47\}, \quad \{57\} \bullet \{08\} = \{8e\}, \quad \{57\} \bullet \{10\} = \{07\}.$$

또한 $\{13\} = \{10\} \oplus \{02\} \oplus \{01\}$ 이므로

$$\{57\} \bullet \{13\} = \{57\} \oplus \{ae\} \oplus \{07\} = \{fe\}. \quad (4.7)$$

4.3 Multiplication of Words by a Fixed Matrix

Input Word $[b_0, b_1, b_2, b_3]$ 와 Word $[a_0, a_1, a_2, a_3]$ 로 정의되는 fixed matrix의 곱은

$$d_0 = (a_0 \bullet b_0) \oplus (a_3 \bullet b_1) \oplus (a_2 \bullet b_2) \oplus (a_1 \bullet b_3) \quad (1)$$

$$d_1 = (a_1 \bullet b_0) \oplus (a_0 \bullet b_1) \oplus (a_3 \bullet b_2) \oplus (a_2 \bullet b_3) \quad (2)$$

$$d_2 = (a_2 \bullet b_0) \oplus (a_1 \bullet b_1) \oplus (a_0 \bullet b_2) \oplus (a_3 \bullet b_3) \quad (3)$$

$$d_3 = (a_3 \bullet b_0) \oplus (a_2 \bullet b_1) \oplus (a_1 \bullet b_2) \oplus (a_0 \bullet b_3). \quad (4.8)$$

4.4 Multiplicative Inverses in $\text{GF}(2^8)$

$b \neq \{00\}$ 에 대해 multiplicative inverse b^{-1} 는

$$b \bullet b^{-1} = \{01\} \quad (4.10)$$

을 만족하며, $b^{-1} = b^{254}$ 로 계산할 수 있다.

학습 주석

여기서 inverse는 “역수” 의미이다. 일반 실수의 $5^{-1} = 1/5$ 처럼, $\text{GF}(2^8)$ 연산 기준으로 곱했을 때 $\{01\}$ 이 되는 Byte를 뜻한다.

5 Algorithm Specifications

AES-128/192/256의 forward function은 CIPHER(), inverse는 INCIPHER()이다. KEYEXPANSION()은 원래 Key로부터 Round Key들을 생성한다.

Algorithm	N_k (words/bits)	N_b (words/bits)	N_r
AES-128	4 / 128	4 / 128	10
AES-192	6 / 192	4 / 128	12
AES-256	8 / 256	4 / 128	14

$$\text{AES-128}(in, key) = \text{CIPHER}(in, 10, \text{KEYEXPANSION}(key)) \quad (4)$$

$$\text{AES-192}(in, key) = \text{CIPHER}(in, 12, \text{KEYEXPANSION}(key)) \quad (5)$$

$$\text{AES-256}(in, key) = \text{CIPHER}(in, 14, \text{KEYEXPANSION}(key)). \quad (5.1)$$

학습 주석

Round 수는 Key length에 따라 10/12/14로 달라진다.

5.1 CIPHER()

한 Round는 SUBBYTES(), SHIFTRROWS(), MIXCOLUMNS(), ADDROUNDKEY()로 구성된다. Initial 단계에는 ADDROUNDKEY()만 수행하고, final Round에는 MIXCOLUMNS()가 없다.

```

procedure CIPHER(in, Nr, w)
  state <- in
  state <- ADDROUNDKEY(state, w[0..3])
  for round from 1 to Nr - 1 do
    state <- SUBBYTES(state)
    state <- SHIFTRROWS(state)
    state <- MIXCOLUMNS(state)
    state <- ADDROUNDKEY(state, w[4*round .. 4*round+3])
  end for
  state <- SUBBYTES(state)
  state <- SHIFTRROWS(state)
  state <- ADDROUNDKEY(state, w[4*Nr .. 4*Nr+3])
  return state
end procedure

```

학습 주석

Final Round에는 MIXCOLUMNS()가 없다. RTL 구현에서 반드시 분기해야 하는 부분이다.

5.1.1 SUBBYTES()

SUBBYTES()는 State의 각 Byte에 AES S-box를 독립적으로 적용하는 invertible, non-linear transformation이다. Input Byte b 에 대해

$$\tilde{b} = \begin{cases} \{00\}, & b = \{00\}, \\ b^{-1}, & b \neq \{00\}, \end{cases} \quad (5.2)$$

이고, affine transformation은

$$b'_i = \tilde{b}_i \oplus \tilde{b}_{(i+4) \bmod 8} \oplus \tilde{b}_{(i+5) \bmod 8} \oplus \tilde{b}_{(i+6) \bmod 8} \oplus \tilde{b}_{(i+7) \bmod 8} \oplus c_i. \quad (5.3)$$

예: $\{53\} \mapsto \{ed\}$.

학습 주석

FPGA에서는 S-box를 LUT/ROM으로 구현하거나 composite field arithmetic으로 계산할 수 있다. 처음 bit-exact 검증에서는 lookup table 방식이 가장 단순하다.

Table 4. SBOX(): input Byte xy 에 대한 substitution value (hexadecimal)

$x \backslash y$	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

5.1.2 SHIFTRROWS()

Row index r 에 대해

$$s'_{r,c} = s_{r,(c+r) \bmod 4}, \quad 0 \leq r < 4, 0 \leq c < 4. \quad (5.5)$$

즉 row 0/1/2/3은 각각 왼쪽으로 0/1/2/3 Byte cyclic shift된다.

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} \rightarrow \begin{bmatrix} a & b & c & d \\ f & g & h & e \\ k & l & i & j \\ p & m & n & o \end{bmatrix}$$

5.1.3 MIXCOLUMNS()

각 State column에 다음 fixed matrix를 GF(2⁸) arithmetic으로 곱한다.

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}. \quad (5.7)$$

예를 들어

$$s'_{0,c} = (\{02\} \bullet s_{0,c}) \oplus (\{03\} \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}. \quad (5.8)$$

5.1.4 ADDROUNDKEY()

Round Key와 State를 bitwise XOR한다.

$$[s'_{0,c}, s'_{1,c}, s'_{2,c}, s'_{3,c}] = [s_{0,c}, s_{1,c}, s_{2,c}, s_{3,c}] \oplus w[4 \cdot \text{round} + c]. \quad (5.9)$$

학습 주석

XOR은 같은 값을 두 번 적용하면 사라지므로 ADDROUNDKEY()는 자기 자신이 inverse이다: $A \oplus K \oplus K = A$.

5.2 KEYEXPANSION()

KEYEXPANSION()은 Key에서 $4(N_r + 1)$ 개의 Word를 생성한다. Round constant는 다음과 같다.

j	Rcon[j]	j	Rcon[j]
1	01000000	6	20000000
2	02000000	7	40000000
3	04000000	8	80000000
4	08000000	9	1b000000
5	10000000	10	36000000

$$\text{ROTWORD}([a_0, a_1, a_2, a_3]) = [a_1, a_2, a_3, a_0], \quad (5.10)$$

$$\text{SUBWORD}([a_0, a_1, a_2, a_3]) = [\text{SBOX}(a_0), \text{SBOX}(a_1), \text{SBOX}(a_2), \text{SBOX}(a_3)]. \quad (5.11)$$

```

procedure KEYEXPANSION(key)
  i <- 0
  while i <= Nk - 1 do
    w[i] <- key[4*i .. 4*i+3]
    i <- i + 1
  end while
  while i <= 4*Nr + 3 do
    temp <- w[i-1]
    if i mod Nk = 0 then
      temp <- SUBWORD(ROTWORD(temp)) XOR Rcon[i/Nk]
    else if Nk > 6 and i mod Nk = 4 then
      temp <- SUBWORD(temp)
    end if
    w[i] <- w[i-Nk] XOR temp
    i <- i + 1
  end while
  return w
end procedure

```

학습 주석

AES-128은 총 44 Word = 176 Byte의 Key schedule을 만든다. 원래 16 Byte Key가 11개의 16 Byte Round Key로 확장된다.

5.3 INVCIPHER()

INVCIPHER()는 CIPHER() transformation을 inverse로 바꾸고 reverse order로 수행한다.

```

procedure INVCIPHER(in, Nr, w)
  state <- in
  state <- ADDROUNDKEY(state, w[4*Nr .. 4*Nr+3])
  for round from Nr - 1 downto 1 do
    state <- INVSHIFTRROWS(state)
    state <- INVSUBBYTES(state)
    state <- ADDROUNDKEY(state, w[4*round .. 4*round+3])
    state <- INVMIXCOLUMNS(state)
  end for
  state <- INVSHIFTRROWS(state)
  state <- INVSUBBYTES(state)
  state <- ADDROUNDKEY(state, w[0..3])
  return state
end procedure

```

5.3.1 INVSHIFTRROWS()

$$s'_{r,c} = s_{r,(c-r) \bmod 4}. \quad (5.12)$$

즉 row 0은 그대로, row 1/2/3은 각각 오른쪽으로 1/2/3칸 cyclic shift한다.

5.3.2 INVSUBBYTES()

INVSUBBYTES()는 State의 각 Byte에 inverse S-box인 INVSBOX()를 적용한다.

Table 6. INVSBOX(): input Byte xy 에 대한 inverse substitution value (hexadecimal)

$x \backslash y$	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
1	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
2	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
3	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
4	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
5	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
6	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
7	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
8	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
9	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
a	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
b	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
c	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
d	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
e	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
f	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

5.3.3 INVMIXCOLUMNS()

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}. \quad (5.14)$$

5.3.4 Inverse of ADDROUNDKEY()

ADDROUNDKEY()는 XOR를 사용하므로 자기 자신이 inverse이다.

5.3.5 EQINVCIPHER()

AES의 algebraic property를 이용하면 CIPHER()와 유사한 structure로 inverse를 표현할 수 있다. 이를 equivalent inverse cipher라고 하며 modified Key schedule dw 를 사용한다.

```
procedure EQINVCIPHER(in, Nr, dw)
  state <- in
  state <- ADDROUNDKEY(state, dw[4*Nr .. 4*Nr+3])
  for round from Nr - 1 downto 1 do
    state <- INVSUBBYTES(state)
    state <- INVSHIFTRROWS(state)
    state <- INVMIXCOLUMNS(state)
    state <- ADDROUNDKEY(state, dw[4*round .. 4*round+3])
  end for
  state <- INVSUBBYTES(state)
  state <- INVSHIFTRROWS(state)
  state <- ADDROUNDKEY(state, dw[0..3])
  return state
end procedure
```

학습 주석

software/FPGA에서 encryption/decryption datapath를 얼마나 공유할지 설계할 때 equivalent inverse cipher 구조가 도움이 될 수 있다. 처음에는 표준 INVCIPHER() 순서를 먼저 정확히 이해하는 것이 좋다.

6 Implementation Considerations

6.1 Key Length Requirements

AES implementation은 128, 192, 256 bit Key length 중 적어도 하나를 지원해야 한다.

6.2 Keying Restrictions

Cryptographic Key가 적절히 생성되었다면 이 표준은 AES 사용에 추가 제한을 두지 않는다.

6.3 Parameter Extensions

현 표준은 N_k , N_b , N_r 의 허용값을 명시한다. 향후 revision을 고려해 구현을 유연하게 설계할 수 있다.

6.4 Implementation Suggestions Regarding Various Platforms

같은 input Key와 data에 대해 표준과 동일한 output을 낸다면 내부 구현 방식이 달라도 equivalent AES implementation이다. 다만 performance-oriented implementation이 side-channel attack까지 자동 방어하는 것은 아니다. computation time, cache behavior, fault injection 등으로 Key-dependent information이 leak될 수 있다.

6.5 Modes of Operation

Block cipher mode of operation은 AES 같은 block cipher를 사용해 confidentiality, authentication 등의 service를 제공한다. NIST-recommended mode는 SP 800-38 series에 규정되어 있다.

학습 주석

AES-XTS는 저장장치용 mode of operation이다. AES primitive 위에 XTS 구조가 추가된다.

References

원문 reference의 title, author, identifier는 그대로 유지한다.

- [1] James Nechvatal et al., Report on the Development of the Advanced Encryption Standard (AES), 2001.
- [2] Joan Daemen and Vincent Rijmen, AES Proposal: Rijndael Document Version 2, 1999.
- [3] Joan Daemen and Vincent Rijmen, The Design of Rijndael, 2nd ed., 2020.
- [4] Michael Artin, Algebra, 2017.
- [5] Menezes et al., Handbook of Applied Cryptography, 1997.
- [6] NIST SP 800-133 Rev.2, Recommendation for Cryptographic Key Generation.

Appendix A - Key Expansion Examples

이 Appendix는 각 Key size에 대한 Key schedule 전개과정의 intermediate value를 보여준다. index i 를 제외한 값은 hexadecimal이다.

A.1 Expansion of a 128-bit Key

Key = 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c

$N_k = 4$, $N_r = 10$. 처음 4 Word는 Key 자체이다.

w0=2b7e1516 w1=28aed2a6 w2=abf71588 w3=09cf4f3c

AES-128 Key expansion intermediate values

i	temp	ROTWORD	SUBWORD	Rcon	변환 후 temp	$w[i - N_k]$	$w[i]$
4	09cf4f3c	cf4f3c09	8a84eb01	01000000	8b84eb01	2b7e1516	a0fafe17
5	a0fafe17	—	—	—	a0fafe17	28aed2a6	88542cb1
6	88542cb1	—	—	—	88542cb1	abf71588	23a33939
7	23a33939	—	—	—	23a33939	09cf4f3c	2a6c7605
8	2a6c7605	6c76052a	50386be5	02000000	52386be5	a0fafe17	f2c295f2
9	f2c295f2	—	—	—	f2c295f2	88542cb1	7a9eb943
10	7a9eb943	—	—	—	7a9eb943	23a33939	5935807a
11	5935807a	—	—	—	5935807a	2a6c7605	7339667f
12	7339667f	59fe7f73	cb42d28f	04000000	cf42d28f	f2c295f2	3d80477d
13	3d80477d	—	—	—	3d80477d	7a9eb943	4716fe3e
14	4716fe3e	—	—	—	4716fe3e	5935807a	1e237e44
15	1e237e44	—	—	—	1e237e44	7339667f	6d7a883b
16	6d7a883b	7a883bd6	dac4e23c	08000000	d2a4e23c	3d80477d	ef44a541
17	ef44a541	—	—	—	ef44a541	4716fe3e	a8525b7f
18	a8525b7f	—	—	—	a8525b7f	1e237e44	b671253b
19	b671253b	—	—	—	b671253b	6d7a883b	db0bad00
20	db0bad00	0bad00db	2b9563b9	10000000	3b9563b9	ef44a541	d4d1ce6f
21	d4d1ce6f	—	—	—	d4d1ce6f	a8525b7f	7c839d87
22	7c839d87	—	—	—	7c839d87	b671253b	caf2b8bc
23	caf2b8bc	—	—	—	caf2b8bc	db0bad00	11f915bc
24	11f915bc	f915bc11	99596582	20000000	b9596582	d4d1ce6f	6d88a37a
25	6d88a37a	—	—	—	6d88a37a	7c839d87	110b3efd
26	110b3efd	—	—	—	110b3efd	caf2b8bc	dbf98641
27	dbf98641	—	—	—	dbf98641	11f915bc	ca0093fd
28	ca0093fd	0093fdca	63dc5474	40000000	23dc5474	6d88a37a	4e5470e
29	4e5470e	—	—	—	4e5470e	110b3efd	5f5fc9f3
30	5f5fc9f3	—	—	—	5f5fc9f3	dbf98641	84a64fb2
31	84a64fb2	—	—	—	84a64fb2	ca0093fd	4ea6dc4f
32	4ea6dc4f	a6dc4fae	2486842f	80000000	a486842f	4e5470e	ead27321
33	ead27321	—	—	—	ead27321	5f5fc9f3	b58dbad2
34	b58dbad2	—	—	—	b58dbad2	84a64fb2	312bf560
35	312bf560	—	—	—	312bf560	4ea6dc4f	7f8d292f
36	7f8d292f	8d2927f7	5da515d2	1b000000	46a515d2	ead27321	ac7766f3
37	ac7766f3	—	—	—	ac7766f3	b58dbad2	19fadc21
38	19fadc21	—	—	—	19fadc21	312bf560	28d12941
39	28d12941	—	—	—	28d12941	7f8d292f	575c006e

<i>i</i>	temp	ROTWORD	SUBWORD	Rcon	변환 후 temp	$w[i - N_k]$	$w[i]$
40	575c006e	5c006e57	4a639f5b	36000000	7c639f5b	ac7766f3	d014f9a8
41	d014f9a8	—	—	—	d014f9a8	19fadc21	c9ec2589
42	c9ec2589	—	—	—	c9ec2589	28d12941	e13f0cc8
43	e13f0cc8	—	—	—	e13f0cc8	575c006e	b6630ca6

학습 주석

Round Key 0은 $w_0 \dots w_3$, Round Key 1은 $w_4 \dots w_7$, ..., Round Key 10은 $w_{40} \dots w_{43}$ 이다.

A.2 Expansion of a 192-bit Key

Key = 8e 73 b0 f7 da 0e 64 52 c8 10 f3 2b 80 90 79 e5 62 f8 ea d2 52 2c 6b 7b

$N_k = 6$, $N_r = 12$.

w0=8e73b0f7 w1=da0e6452 w2=c810f32b w3=809079e5 w4=62f8ead2 w5=522c6b7b

AES-192 Key expansion intermediate values

<i>i</i>	temp	ROTWORD	SUBWORD	Rcon	변환 후 temp	$w[i - N_k]$	$w[i]$
6	522c6b7b	2c6b7b52	717f2100	01000000	707f2100	8c73b0f7	60c91f7
7	60c91f7	—	—	—	60c91f7	da0e6452	2402f5a5
8	2402f5a5	—	—	—	2402f5a5	c810f32b	ec12068e
9	ec12068e	—	—	—	ec12068e	809079e5	6c827f6b
10	6c827f6b	—	—	—	6c827f6b	62f8ead2	0e7a95b9
11	0e7a95b9	—	—	—	0e7a95b9	522c6b7b	5c56fec2
12	5c56fec2	56fec25c	b1bb254a	02000000	60c91f7	4db7b4bd	—
13	4db7b4bd	—	—	—	4db7b4bd	2402f5a5	69b54118
14	69b54118	—	—	—	69b54118	ec12068e	85a74796
15	85a74796	—	—	—	85a74796	6c827f6b	e92538fd
16	e92538fd	—	—	—	e92538fd	0e7a95b9	e75fad44
17	e75fad44	—	—	—	e75fad44	5c56fec2	b0995386
18	b0995386	095386bb	01ed44ea	04000000	05ed44ea	4db7b4bd	485af057
19	485af057	—	—	—	485af057	69b54118	21efb14f
20	21efb14f	—	—	—	21efb14f	85a74796	a448fed9
21	a448fed9	—	—	—	a448fed9	e92538fd	4d6dce24
22	4d6dce24	—	—	—	4d6dce24	e75fad44	aa326360
23	aa326360	—	—	—	aa326360	b0995386	113b306e
24	113b306e	3b30e611	e2048e82	08000000	ea048e82	485af057	a25e7ed5
25	a25e7ed5	—	—	—	a25e7ed5	21efb14f	83b1cf9a
26	83b1cf9a	—	—	—	83b1cf9a	a448fed9	27f93943
27	27f93943	—	—	—	27f93943	4d6dce24	6a94767
28	6a94767	—	—	—	6a94767	aa326360	0ba69407
29	0ba69407	—	—	—	0ba69407	113b306e	d19da4e1
30	d19da4e1	9da4e1d1	5e49f83e	10000000	4e49f83e	a25e7ed5	ec1786eb
31	ec1786eb	—	—	—	ec1786eb	83b1cf9a	6fa64971
32	6fa64971	—	—	—	6fa64971	27f93943	485f7032
33	485f7032	—	—	—	485f7032	6a94767	22cb8755
34	22cb8755	—	—	—	22cb8755	0ba69407	e26d1352
35	e26d1352	—	—	—	e26d1352	d19da4e1	330b7b3b
36	330b7b3b	f0b7b333	8ca96dc3	20000000	aca96dc3	ec1786eb	40beeb28
37	40beeb28	—	—	—	40beeb28	6fa64971	2f18a259
38	2f18a259	—	—	—	2f18a259	485f7032	6747d26b
39	6747d26b	—	—	—	6747d26b	22cb8755	458c553e
40	458c553e	—	—	—	458c553e	e26d1352	a7e1466e
41	a7e1466e	—	—	—	a7e1466e	330b7b3b	9411f1df
42	9411f1df	11f1df94	82a19e22	40000000	c2a19e22	40beeb28	821f750a
43	821f750a	—	—	—	821f750a	2f18a259	ad07d753
44	ad07d753	—	—	—	ad07d753	6747d26b	ca400538
45	ca400538	—	—	—	ca400538	458c553e	8fcc5006
46	8fcc5006	—	—	—	8fcc5006	a7e1466e	282d166a
47	282d166a	—	—	—	282d166a	9411f1df	bc3ce7b5
48	bc3ce7b5	3ce7b5bc	eb94d565	80000000	6b94d565	821f750a	e98ba06f
49	e98ba06f	—	—	—	e98ba06f	ad07d753	448c773c
50	448c773c	—	—	—	448c773c	ca400538	8ecc7204
51	8ecc7204	—	—	—	8ecc7204	8fcc5006	01002202

A.3 Expansion of a 256-bit Key

Key = 60 3d eb 10 15 ca 71 be 2b 73 ae f0 85 7d 77 81 1f 35 2c 07 3b 61 08 d7 2d 98 10 a3 09 14 df f4

$N_k = 8$, $N_r = 14$.

w0=603deb10 w1=15ca71be w2=2b73aef0 w3=857d7781

w4=1f352c07 w5=3b6108d7 w6=2d9810a3 w7=0914dff4

AES-256 Key expansion intermediate values

<i>i</i>	temp	ROTWORD	SUBWORD	Rcon	변환 후 temp	$w[i - N_k]$	$w[i]$
8	0914dff4	14dff409	fa9ebf01	01000000	fb9ebf01	603deb10	9ba35411
9	9ba35411	—	—	—	9ba35411	15ca71be	8e6925af
10	8e6925af	—	—	—	8e6925af	2b73aef0	a51a8b5f
11	a51a8b5f	—	—	—	a51a8b5f	857d7781	2067fde
12	2067fde	—	—	—	b785b01d	1352c07	a8b09c1a
13	a8b09c1a	—	—	—	a8b09c1a	3b6108d7	93d194cd
14	93d194cd	—	—	—	93d194cd	2d9810a3	be49846e
15	be49846e	—	—	—	be49846e	0914dff4	b75d5b9a
16	b75d5b9a	5d5b9ab7	4c39b8a9	02000000	4c39b8a9	9ba35411	d59aecb8
17	d59aecb8	—	—	—	d59aecb8	8e6925af	5bf3c917
18	5bf3c917	—	—	—	5bf3c917	a51a8b5f	fee94248

<i>i</i>	temp	ROTWORD	SUBWORD	Rcon	변환 후 temp	$w[i - N_k]$	$w[i]$
19	fee94248	—	—	—	fee94248	2067fde	de8ebe9e
20	de8ebe9e	—	—	—	1d19ae90	a8b09c1a	b5a9328a
21	b5a9328a	—	—	—	b5a9328a	93d194cd	2678ae47
22	2678ae47	—	—	—	2678ae47	be49846e	98312229
23	98312229	—	—	—	98312229	b75d5b9a	2f6c79b3
24	2f6c79b3	6c79b32f	50b66d15	04000000	54b66d15	d59aecb8	812c81ad
25	812c81ad	—	—	—	812c81ad	5bf3c917	dadf48ba
26	dadf48ba	—	—	—	dadf48ba	fee94248	24360af2
27	24360af2	—	—	—	24360af2	de8ebe9e	fab8b464
28	fab8b464	—	—	—	2d6c8d43	b5a9328a	98c5bfc9
29	98c5bfc9	—	—	—	98c5bfc9	2678ae47	bebd198e
30	bebd198e	—	—	—	bebd198e	98312229	268c3ba7
31	268c3ba7	—	—	—	268c3ba7	2f6c79b3	09e04214
32	09e04214	e0421409	e12cfa01	08000000	e92cfa01	812c81ad	68007bac
33	68007bac	—	—	—	68007bac	dadf48ba	b2df3316
34	b2df3316	—	—	—	b2df3316	24360af2	96e939e4
35	96e939e4	—	—	—	96e939e4	fab8b464	6c518d80
36	6c518d80	—	—	—	50d15dcd	98c5bfc9	c814e204
37	c814e204	—	—	—	c814e204	bebd198e	76a9fb8a
38	76a9fb8a	—	—	—	76a9fb8a	268c3ba7	5025c02d
39	5025c02d	—	—	—	5025c02d	09e04214	59c38239
40	59c38239	c5823959	a61312cb	10000000	b61312cb	68007bac	de136967
41	de136967	—	—	—	de136967	b2df3316	6ccc5a71
42	6ccc5a71	—	—	—	6ccc5a71	96e939e4	fa256395
43	fa256395	—	—	—	fa256395	6c518d80	9674ee15
44	9674ee15	—	—	—	90922859	c814e204	5886ca5d
45	5886ca5d	—	—	—	5886ca5d	76a9fb8a	2e2f31d7
46	2e2f31d7	—	—	—	2e2f31d7	5025c02d	7e0af1fa
47	7e0af1fa	—	—	—	7e0af1fa	59c58239	27c7f3c3
48	27c7f3c3	cf73c327	8a8f2ecc	20000000	aa8f2ecc	de136967	749c47ab
49	749c47ab	—	—	—	749c47ab	6ccc5a71	18501dda
50	18501dda	—	—	—	18501dda	fa256395	e2757e4f
51	e2757e4f	—	—	—	e2757e4f	9674ee15	7401905a
52	7401905a	—	—	—	927c60be	5886ca5d	cafaa3c3
53	cafaa3c3	—	—	—	cafaa3c3	2e2f31d7	e4d59b34
54	e4d59b34	—	—	—	e4d59b34	7e0af1fa	9adf6ace
55	9adf6ace	—	—	—	9adf6ace	27c7f3c3	bd10190d
56	bd10190d	10190dbd	cad4d77a	40000000	8ad4d77a	749c47ab	f64990d1
57	f64990d1	—	—	—	f64990d1	18501dda	ee188d0b
58	ee188d0b	—	—	—	ee188d0b	e2757e4f	046df344
59	046df344	—	—	—	046df344	7401905a	706c631e

Appendix B - Cipher Example

이 example은 Block length와 Key length가 모두 16 Byte인 AES-128에서 State가 Round별로 어떻게 변하는지 보여준다.

Input = 32 43 f6 a8 88 5a 30 8d 31 31 98 a2 e0 37 07 34

Key = 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c

Round Key는 Appendix A의 AES-128 Key Expansion 결과를 사용한다. Matrix는 FIPS 197의 column-major State ordering을 따른다.

Round 1

Start of Round	After SUBBYTES	After SHIFTRROWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix}$	$\begin{bmatrix} d4 & e0 & b8 & 1e \\ 27 & bf & b4 & 41 \\ 11 & 98 & 5d & 52 \\ ae & f1 & e5 & 30 \end{bmatrix}$	$\begin{bmatrix} d4 & e0 & b8 & 1e \\ bf & b4 & 41 & 27 \\ 5d & 52 & 11 & 98 \\ 30 & ae & f1 & e5 \end{bmatrix}$	$\begin{bmatrix} 04 & e0 & 48 & 28 \\ 66 & cb & f8 & 06 \\ 81 & 19 & d3 & 26 \\ e5 & 9a & 7a & 4c \end{bmatrix}$	$\begin{bmatrix} a0 & 88 & 23 & 2a \\ fa & 54 & a3 & 6c \\ fe & 2c & 39 & 76 \\ 17 & b1 & 39 & 05 \end{bmatrix}$

Round 2

Start of Round	After SUBBYTES	After SHIFTRROWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} a4 & 68 & 6b & 02 \\ 9c & 9f & 5b & 6a \\ 7f & 35 & ea & 50 \\ f2 & 2b & 43 & 49 \end{bmatrix}$	$\begin{bmatrix} 49 & 45 & 7f & 77 \\ de & db & 39 & 02 \\ d2 & 96 & 87 & 53 \\ 89 & f1 & 1a & 3b \end{bmatrix}$	$\begin{bmatrix} 49 & 45 & 7f & 77 \\ db & 39 & 02 & de \\ 87 & 53 & d2 & 96 \\ 3b & 89 & f1 & 1a \end{bmatrix}$	$\begin{bmatrix} 58 & 1b & db & 1b \\ 4d & 4b & e7 & 6b \\ ca & 5a & ca & b0 \\ f1 & ac & a8 & e5 \end{bmatrix}$	$\begin{bmatrix} f2 & 7a & 59 & 73 \\ c2 & 96 & 35 & 59 \\ 95 & b9 & 80 & f6 \\ f2 & 43 & 7a & 7f \end{bmatrix}$

Round 3

Start of Round	After SUBBYTES	After SHIFTRROWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} aa & 61 & 82 & 68 \\ 8f & dd & d2 & 32 \\ 5f & e3 & 4a & 46 \\ 03 & ef & d2 & 9a \end{bmatrix}$	$\begin{bmatrix} ac & ef & 13 & 45 \\ 73 & c1 & b5 & 23 \\ cf & 11 & d6 & 5a \\ 7b & df & b5 & b8 \end{bmatrix}$	$\begin{bmatrix} ac & ef & 13 & 45 \\ c1 & b5 & 23 & 73 \\ d6 & 5a & cf & 11 \\ b8 & 7b & df & b5 \end{bmatrix}$	$\begin{bmatrix} 75 & 20 & 53 & bb \\ ec & 0b & c0 & 25 \\ 09 & 63 & cf & d0 \\ 93 & 33 & 7c & dc \end{bmatrix}$	$\begin{bmatrix} 3d & 47 & 1e & 6d \\ 80 & 16 & 23 & 7a \\ 47 & fe & 7e & 88 \\ 7d & 3e & 44 & 3b \end{bmatrix}$

Round 4

Start of Round	After SUBBYTES	After SHIFTRWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} 48 & 67 & 4d & d6 \\ 6c & 1d & e3 & 5f \\ 4e & 9d & b1 & 58 \\ ee & 0d & 38 & e7 \end{bmatrix}$	$\begin{bmatrix} 52 & 85 & e3 & f6 \\ 50 & a4 & 11 & cf \\ 2f & 5e & c8 & 6a \\ 28 & d7 & 07 & 94 \end{bmatrix}$	$\begin{bmatrix} 52 & 85 & e3 & f6 \\ a4 & 11 & cf & 50 \\ c8 & 6a & 2f & 5e \\ 94 & 28 & d7 & 07 \end{bmatrix}$	$\begin{bmatrix} 0f & 60 & 6f & 5e \\ d6 & 31 & c0 & b3 \\ da & 38 & 10 & 13 \\ a9 & bf & 6b & 01 \end{bmatrix}$	$\begin{bmatrix} ef & a8 & b6 & db \\ 44 & 52 & 71 & 0b \\ a5 & 5b & 25 & ad \\ 41 & 7f & 3b & 00 \end{bmatrix}$

Round 5

Start of Round	After SUBBYTES	After SHIFTRWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} e0 & c8 & d9 & 85 \\ 92 & 63 & b1 & b8 \\ 7f & 63 & 35 & be \\ e8 & c0 & 50 & 01 \end{bmatrix}$	$\begin{bmatrix} e1 & e8 & 35 & 97 \\ 4f & fb & c8 & 6c \\ d2 & fb & 96 & ae \\ 9b & ba & 53 & 7c \end{bmatrix}$	$\begin{bmatrix} e1 & e8 & 35 & 97 \\ fb & c8 & 6c & 4f \\ 96 & ae & d2 & fb \\ 7c & 9b & ba & 53 \end{bmatrix}$	$\begin{bmatrix} 25 & bd & b6 & 4c \\ d1 & 11 & 3a & 4c \\ a9 & d1 & 33 & c0 \\ ad & 68 & 8e & b0 \end{bmatrix}$	$\begin{bmatrix} d4 & 7c & ca & 11 \\ d1 & 83 & f2 & f9 \\ c6 & 9d & b8 & 15 \\ f8 & 87 & bc & bc \end{bmatrix}$

Round 6

Start of Round	After SUBBYTES	After SHIFTRWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} f1 & c1 & 7c & 5d \\ 00 & 92 & c8 & b5 \\ 6f & 4c & 8b & d5 \\ 55 & ef & 32 & 0c \end{bmatrix}$	$\begin{bmatrix} a1 & 78 & 10 & 4c \\ 63 & 4f & e8 & d5 \\ a8 & 29 & 3d & 03 \\ fc & df & 23 & fe \end{bmatrix}$	$\begin{bmatrix} a1 & 78 & 10 & 4c \\ 4f & e8 & d5 & 63 \\ 3d & 03 & a8 & 29 \\ fe & fc & df & 23 \end{bmatrix}$	$\begin{bmatrix} 4b & 2c & 33 & 37 \\ 86 & 4a & 9d & d2 \\ 8d & 89 & f4 & 18 \\ 6d & 80 & e8 & d8 \end{bmatrix}$	$\begin{bmatrix} 6d & 11 & db & ca \\ 88 & 0b & f9 & 00 \\ a3 & 3e & 86 & 93 \\ 7a & fd & 41 & fd \end{bmatrix}$

Round 7

Start of Round	After SUBBYTES	After SHIFTRWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} 26 & 3d & e8 & fd \\ 0e & 41 & 64 & d2 \\ 2e & b7 & 72 & 8b \\ 17 & 7d & a9 & 25 \end{bmatrix}$	$\begin{bmatrix} f7 & 27 & 9b & 54 \\ ab & 83 & 43 & b5 \\ 31 & a9 & 40 & 3d \\ f0 & ff & d3 & 3f \end{bmatrix}$	$\begin{bmatrix} f7 & 27 & 9b & 54 \\ 83 & 43 & b5 & ab \\ 40 & 3d & 31 & a9 \\ 3f & f0 & ff & d3 \end{bmatrix}$	$\begin{bmatrix} 14 & 46 & 27 & 34 \\ 15 & 16 & 46 & 2a \\ b5 & 15 & 56 & d8 \\ bf & ec & d7 & 43 \end{bmatrix}$	$\begin{bmatrix} 4e & 5f & 84 & 4e \\ 54 & 5f & a6 & a6 \\ f7 & c9 & 4f & dc \\ 0e & f3 & b2 & 4f \end{bmatrix}$

Round 8

Start of Round	After SUBBYTES	After SHIFTRWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} 5a & 19 & a3 & 7a \\ 41 & 49 & e0 & 8c \\ 42 & dc & 19 & 04 \\ b1 & 1f & 65 & 0c \end{bmatrix}$	$\begin{bmatrix} be & d4 & 0a & da \\ 83 & 3b & e1 & 64 \\ 2c & 86 & d4 & f2 \\ c8 & c0 & 4d & fe \end{bmatrix}$	$\begin{bmatrix} be & d4 & 0a & da \\ 3b & e1 & 64 & 83 \\ d4 & f2 & 2c & 86 \\ fe & c8 & c0 & 4d \end{bmatrix}$	$\begin{bmatrix} 00 & b1 & 54 & fa \\ 51 & c8 & 76 & 1b \\ 2f & 89 & 6d & 99 \\ d1 & ff & cd & ea \end{bmatrix}$	$\begin{bmatrix} ea & b5 & 31 & 7f \\ d2 & 8d & 2b & 8d \\ 73 & ba & f5 & 29 \\ 21 & d2 & 60 & 2f \end{bmatrix}$

Round 9

Start of Round	After SUBBYTES	After SHIFTRWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} ea & 04 & 65 & 85 \\ 83 & 45 & 5d & 96 \\ 5c & 33 & 98 & b0 \\ f0 & 2d & ad & c5 \end{bmatrix}$	$\begin{bmatrix} 87 & f2 & 4d & 97 \\ ec & 6e & 4c & 90 \\ 4a & c3 & 46 & e7 \\ 8c & d8 & 95 & a6 \end{bmatrix}$	$\begin{bmatrix} 87 & f2 & 4d & 97 \\ 6e & 4c & 90 & ec \\ 46 & e7 & 4a & c3 \\ a6 & 8c & d8 & 95 \end{bmatrix}$	$\begin{bmatrix} 47 & 40 & a3 & 4c \\ 37 & d4 & 70 & 9f \\ 94 & e4 & 3a & 42 \\ ed & a5 & a6 & bc \end{bmatrix}$	$\begin{bmatrix} ac & 19 & 28 & 57 \\ 77 & fa & d1 & 5c \\ 66 & dc & 29 & 00 \\ f3 & 21 & 41 & 6e \end{bmatrix}$

Round 10

Start of Round	After SUBBYTES	After SHIFTRWS	Round Key
$\begin{bmatrix} eb & 59 & 8b & 1b \\ 40 & 2e & a1 & c3 \\ f2 & 38 & 13 & 42 \\ 1e & 84 & e7 & d2 \end{bmatrix}$	$\begin{bmatrix} e9 & cb & 3d & af \\ 09 & 31 & 32 & 2e \\ 89 & 07 & 7d & 2c \\ 72 & 5f & 94 & b5 \end{bmatrix}$	$\begin{bmatrix} e9 & cb & 3d & af \\ 31 & 32 & 2e & 09 \\ 7d & 2c & 89 & 07 \\ b5 & 72 & 5f & 94 \end{bmatrix}$	$\begin{bmatrix} d0 & c9 & e1 & b6 \\ 14 & ee & 3f & 63 \\ f9 & 25 & 0c & 0c \\ a8 & 89 & c8 & a6 \end{bmatrix}$

Output = 39 25 84 1d 02 dc 09 fb dc 11 85 97 19 6a 0b 32.

학습 주석

최종 ciphertext만 비교하면 오류가 어느 Round에서 발생했는지 알기 어렵다. 이 Appendix의 intermediate State를 비교하면 SUBBYTES/SHIFTRWS/MIXCOLUMNS/ADDDROUNDKEY 중 어느 단계에서 틀렸는지 빠르게 좁힐 수 있다.

Appendix C - Example Vectors

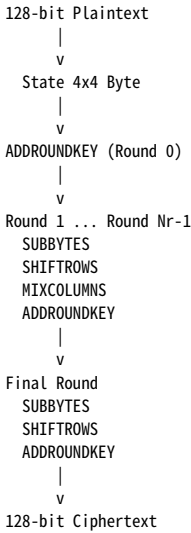
NIST Computer Security Resource Center는 AES-128, AES-192, AES-256에 대해 intermediate value를 포함한 상세 example vector를 제공한다.

Appendix D - Change Log (Informative)

2023 Update 1은 AES의 technical specification 자체를 변경한 것이 아니라 설명, formatting, reference, illustration 및 implementation attack 관련 내용을 정비한 editorial update이다. 주요 변경사항은 다음과 같다.

- publication formatting과 문장을 명확하게 개선했다.
- title page, foreword, abstract, keywords를 추가했다.
- announcement section을 현재 statute, regulation, validation program에 맞게 갱신했다.
- AES-128/192/256을 명시하고 term/function/symbol 목록을 정비했다.
- Byte sequence indexing 및 finite field arithmetic 설명을 개선했다.
- CIPHER, KEYEXPANSION, INVCIPHER pseudocode formatting을 개선했다.
- SHIFTRWS/INVSHIFTRWS 설명을 개선하고 기존 inverse 설명의 오류를 수정했다.
- AES-128/192/256 KEYEXPANSION illustration을 추가했다.
- equivalent inverse cipher의 modified Key expansion routine을 별도 algorithm으로 정리했다.
- implementation attack에 대한 설명을 확대했다.
- Appendix C의 고정 예제를 NIST가 유지관리하는 example vector reference로 대체했다.

학습용 핵심 정리



만드시 기억할 내용

- 1. AES Block size는 항상 128 bit이다.
- 2. AES-128/192/256의 숫자는 Key length이다.
- 3. Round 수는 10/12/14이다.
- 4. State input은 column-major mapping이다.
- 5. SUBBYTES는 non-linear S-box substitution이다.
- 6. SHIFTRWS는 row별 cyclic shift다.
- 7. MIXCOLUMNS는 GF(2⁸) fixed matrix multiplication이다.
- 8. ADDROUNDKEY는 XOR이다.
- 9. Final Round에는 MIXCOLUMNS가 없다.
- 10. KEYEXPANSION은 원래 Key를 N_r + 1개의 Round Key로 확장한다.
- 11. inverse는 원래 변환을 되돌리는 역변환이다.
- 12. 구현은 NIST example vector와 intermediate State를 bit-exact로 비교한다.

English	한국어	의미
plaintext	평문	암호화 전 data
ciphertext	암호문	암호화 후 data
encrypt / encipher	암호화	plaintext → ciphertext
decrypt / decipher	복호화	ciphertext → plaintext
inverse	역변환/역연산	변환 결과를 원래 값으로 되돌리는 반대 연산
state	상태 배열	AES intermediate 16-Byte value
round	라운드	반복되는 transformation 단계
round key	라운드 키	각 Round에서 XOR되는 128-bit Key
key schedule	키 스케줄	전체 Round Key sequence
S-box	치환표	Byte를 다른 Byte로 변환하는 non-linear table
finite field	유한체	element 개수가 유한한 field
Galois Field	갈루아 체	finite field의 다른 표현
affine transformation	아핀 변환	matrix multiplication + vector addition
XOR	배타적 OR	같으면 0, 다르면 1
mode of operation	운용 모드	block cipher를 실제 data 보호 구조로 사용하는 방법